E-Commerce Admin Frontend Architecture and Implementation

Updated for the 72-screen admin scope · March 25, 2026

Executive overview

This document defines the updated **admin frontend architecture** for the e-commerce platform after the admin scope was expanded from a standard operations dashboard to a **72-screen control-heavy admin system**.

The admin is now positioned as the platform's:

- operational control center
- customer intelligence console
- catalog and merchandising workspace
- order, payment, and fulfillment command layer
- service and complaints handling desk
- governance, audit, and investigation workspace
- system observability and exception-monitoring surface

The target is to give admins control over every major workflow while preserving strong traceability, role-based visibility, and confirmation flows for sensitive actions.

Official scope position

The admin frontend should now be treated as:

- **72 core screens** for implementation planning
- **94 route-ready views** when deeper analytics and investigation views are fully split into standalone screens

The 72-screen scope is the official build target.

Architecture goals

The admin frontend must allow authorized staff to control and monitor:

- products, variants, media, merchandising, and product analytics
- categories, brands, and review moderation
- inventory, warehouses, movement history, and low-stock queues
- orders, shipments, returns, cancellations, and fulfillment queues
- payments, refunds, failed payments, and financial exceptions
- customers, account actions, risk profiles, and activity timelines
- support queues, complaint handling, SLA tracking, and support analytics
- banners, CMS pages, coupons, promotions, and campaign performance
- reports across sales, products, inventory, customers, support, and marketing
- audit logs, admin action logs, alerts, security events, incidents, and fraud review
- settings, webhooks, jobs, and integration health

Product design principles

Operational clarity

Every screen should answer at least one of these questions:

- what is happening now?
- what needs action now?
- what changed?
- what is risky?
- who did what?
- what is blocked or failing?

Traceability first

The admin must surface:

- actor
- timestamp
- previous vs new value where relevant
- linked entity context
- approval or confirmation requirements for high-risk actions

High information density without chaos

Use:

- dashboard cards
- queue views
- filter bars
- searchable tables
- summary side panels

- right-side drawers
- deep detail pages
- timelines
- analytics panels

Role-aware control

The architecture must support:

- route-level access control
- action-level permission checks
- hidden modules for unauthorized roles
- disabled or read-only controls when a role can inspect but not mutate

Single-store now, multi-store ready later

The current UI should stay single-store in feel.

There should be no active store switcher now, but the shell should leave room for a future store selector in the header if multi-store is introduced.

Recommended frontend stack

- React
- TypeScript
- React Router
- TanStack React Query
- React Hook Form
- Zod
- Zustand for UI state where needed
- Tailwind CSS
- TanStack Table
- Recharts or equivalent chart library
- toast and modal utilities

App shell architecture

Main shell regions

- header
- primary sidebar
- optional secondary panel area
- main content region

- global search / command palette later
- toast region
- confirm-dialog host
- side-drawer host

Header responsibilities

The header should support:

- page title and breadcrumb
- global search trigger
- quick action launcher
- alert indicator
- urgent support indicator
- profile menu
- security/session menu
- future store-selector slot

Sidebar groups

- Overview
- Catalog
- Inventory
- Orders
- Payments
- Customers
- Support
- Content
- Marketing
- Reports
- Security & Audit
- System

Route architecture by module

A. Access and shell routes

- /admin/login
- /admin/forgot-password
- /admin/reset-password
- /admin/forbidden
- /admin/profile/security

B. Dashboard routes

- /admin/dashboard
- /admin/dashboard/sales
- /admin/dashboard/operations
- /admin/dashboard/support
- /admin/dashboard/risk
- /admin/dashboard/system-health

C. Catalog routes

- /admin/catalog/products
- /admin/catalog/products/new
- /admin/catalog/products/:productId/edit
- /admin/catalog/products/:productId
- /admin/catalog/products/:productId/analytics
- /admin/catalog/products/:productId/variants
- /admin/catalog/products/:productId/media
- /admin/catalog/products/:productId/pricing
- /admin/catalog/products/:productId/inventory
- /admin/catalog/categories
- /admin/catalog/brands
- /admin/catalog/reviews

D. Inventory routes

- /admin/inventory/overview
- /admin/inventory/low-stock
- /admin/inventory/out-of-stock
- /admin/inventory/movements
- /admin/inventory/adjustments
- /admin/inventory/warehouses
- /admin/inventory/warehouses/:warehouseId
- /admin/inventory/warehouses/:warehouseId/inventory

E. Orders and fulfillment routes

- /admin/orders
- /admin/orders/:orderId
- /admin/orders/:orderId/timeline
- /admin/orders/fulfillment-queue
- /admin/orders/dispatch-queue

- /admin/shipments/:shipmentId
- /admin/shipments/:shipmentId/tracking
- /admin/returns
- /admin/returns/:returnId
- /admin/orders/cancellation-requests

F. Payments and finance routes

- /admin/payments
- /admin/payments/:paymentId
- /admin/payments/:paymentId/transactions
- /admin/refunds
- /admin/refunds/:refundId
- /admin/payments/failed-investigations
- /admin/finance/exceptions

G. Customer routes

- /admin/customers
- /admin/customers/:customerId
- /admin/customers/:customerId/activity
- /admin/customers/:customerId/orders
- /admin/customers/:customerId/support
- /admin/customers/:customerId/reviews
- /admin/customers/:customerId/risk
- /admin/customers/:customerId/actions

H. Support routes

- /admin/support/tickets
- /admin/support/tickets/:ticketId
- /admin/support/queue
- /admin/support/pre-purchase
- /admin/support/complaints
- /admin/support/analytics

I. Content and marketing routes

- /admin/content/banners
- /admin/content/pages
- /admin/marketing/coupons
- /admin/marketing/coupons/analytics
- /admin/marketing/promotions

- /admin/marketing/promotions/:promotionId/rules
- /admin/marketing/campaigns/performance

J. Reporting routes

- /admin/reports
- /admin/reports/sales
- /admin/reports/products
- /admin/reports/inventory
- /admin/reports/customers
- /admin/reports/support
- /admin/reports/refunds-returns
- /admin/reports/marketing

K. Security and governance routes

- /admin/security/audit-logs
- /admin/security/admin-actions
- /admin/security/user-activity
- /admin/security/alerts
- /admin/security/alerts/:alertId
- /admin/security/events
- /admin/security/events/:securityEventId
- /admin/security/incidents
- /admin/security/incidents/:incidentId
- /admin/security/risk-signals

L. System routes

- /admin/system/settings
- /admin/system/settings/checkout
- /admin/system/settings/reviews
- /admin/system/settings/support
- /admin/system/webhooks
- /admin/system/jobs
- /admin/system/integrations-health

Major screen families that require dedicated detail views

The architecture now assumes separate deep-detail pages for:

- products
- product analytics

- warehouses
- orders
- order timelines
- shipments
- payments
- refunds
- customers
- customer activity
- customer risk profiles
- support tickets
- alerts
- security events
- incidents

These dedicated views are what make the admin manageable at scale and reduce the need to overload tables or drawers.

Confirmation-dialog architecture

Sensitive actions should use a shared confirmation framework.

Actions that require confirmation

- publish or unpublish product
- archive product
- change product pricing
- create large negative stock adjustments
- cancel order
- override fulfillment state
- approve refund
- reject or close return
- suspend or reactivate customer
- edit sensitive settings
- assign powerful roles
- acknowledge or resolve security alerts
- close incidents
- retry webhook or job processing when allowed

Dialog requirements

Every sensitive dialog should support:

- entity summary
- impact or consequence note
- optional reason field
- confirm and cancel buttons
- async mutation state
- post-success audit capture

Shared UI patterns

- page header
- filter bar
- entity table
- quick statistics cards
- timeline component
- activity feed component
- right-side drawer
- JSON inspector for technical records
- confirmation modal
- destructive action modal
- approval dialog with reason capture
- empty state and no-results state
- loading skeletons
- permission gate wrappers

Data architecture expectations

Each module should follow the same pattern:

- feature-scoped query keys
- server state in React Query
- form state in React Hook Form
- minimal shared global state
- reusable API client layer
- permission helper layer
- shared mutation patterns with confirmation hooks

Recommended frontend folder structure

```
src/
  app/
    router/
    providers/
    layouts/
  components/
    ui/
    tables/
```

```
    filters/
    dialogs/
    timelines/
    charts/
    entity-cards/
  features/
    auth/
    dashboard/
    catalog/
    inventory/
    orders/
    payments/
    customers/
    support/
    content/
    marketing/
    reports/
    security/
    system/
  lib/
    api/
    auth/
    permissions/
    utils/
    constants/
    types/
  hooks/
  state/
```

Implementation phases

Phase 1

- app shell and route protection
- dashboard overview
- products list/detail/create/edit
- categories

n- brands

- orders list/detail

Phase 2

- product analytics and variants/media/pricing/inventory subpages
- inventory overview and queues
- warehouses
- payments list/detail
- customers list/detail/activity

Phase 3

- returns, refunds, failed payment investigation, finance exceptions
- support queues and ticket detail
- content and marketing modules

- reports overview and core report routes

Phase 4

- audit logs, alerts, security events, incidents, risk review
- settings split pages
- webhook, jobs, and integration health monitors
- role-hardening and action-confirmation coverage across all modules

Final architecture position

The admin frontend should now be treated as a **72-screen enterprise-style operations and governance interface**, not as a standard CRUD dashboard.

It is intentionally designed to support:

- business operations
- executive visibility
- customer intelligence
- financial control
- service management
- investigation workflows
- security oversight
- system observability